

Generate and Measure 60 Hz V-Sync in X68000 31 kHz Mode

From the MS.X implementation to a C experiment and emulator improvements

Vertical timing basics · CRTC R04–R07 · V-DISP measurement · Emulator comparison

The standard X68000 mode is about 55 Hz, not the MSX's 60 Hz

X68000 (standard 31 kHz mode)

about 55.5 Hz

vertical-sync events per second

MSX (NTSC mode)

60 Hz

often used as the game-time reference

The MSX development story presents this mismatch as the first obstacle.

"MSX runs at 60 Hz in NTSC mode, whereas the X68000 normally runs at about 55 Hz."

For software paced by vertical sync, the roughly 4.5 Hz difference changes speed and event timing.

[1] English translation from e-book v1.0, PDF p.37 (printed p.35); representative values: PDF p.8 (printed p.6).

MS.X programmed the CRTC directly to create a 60 Hz display mode

Solution described in the book

"It was addressed by directly modifying the X68000 CRTC registers and rebuilding the display mode so that vertical sync became 60 Hz."

What is a CRTC?

A CRT Controller determines the horizontal and vertical sync, blanking, and visible intervals that make up one display frame.

Verification in this guide

1. Program R04-R07 for 525 lines
2. Measure 600 V-DISP periods
3. About 10 seconds means about 60 Hz

[1] English translation: e-book v1.0, PDF p.37 (printed p.35).

Change vertical sync to about 60 Hz while preserving horizontal timing

- 01 First, establish how the CRTC controls vertical sync

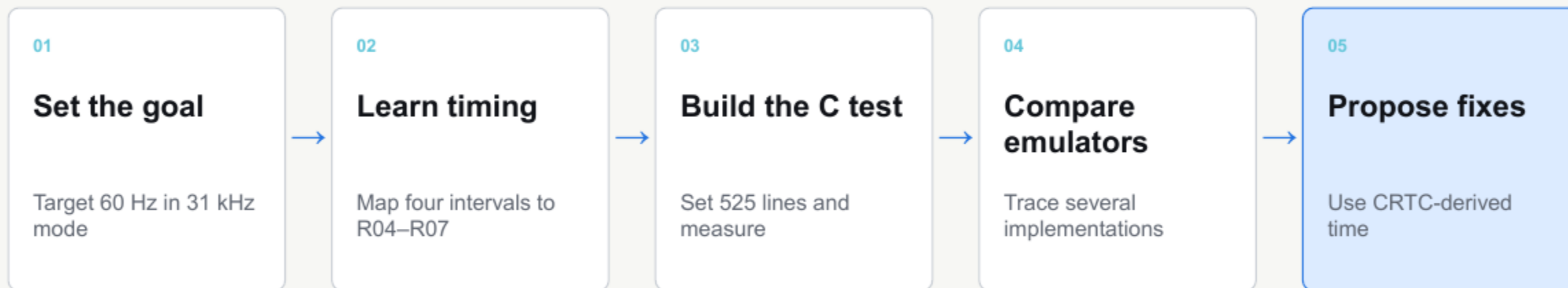
- 02 Then build a C experiment based on MS.X with explicit restoration limits

- 03 Finally, compare measured results with emulator behavior

Measure the configured vertical sync and re-select the saved IOCS mode on exit.

Follow five stages from MS.X to emulator improvements

Understand timing and register meaning before reading values and code.



The first half explains generation and measurement; the second examines how emulators reproduce them.

XEiJ treats R04-R07 as the boundaries of four vertical intervals

R04`crtR04VFrontEndCurr`

end of vertical front porch

R05`crtR05VSyncEndCurr`

end of vertical sync

R06`crtR06VBackEndCurr`

end of vertical back porch

R07`crtR07VDispEndCurr`

end of vertical display

Each register stores a raster boundary, not the interval length itself.

[2] XEiJ / CRTC.java: vertical timing diagram and R04-R07 Current fields.

The official MS.X source maps R04-R07 to the CRTC hardware addresses

```
#define CRTR_04 (*(volatile uint16_t*)0xe80008)
#define CRTR_05 (*(volatile uint16_t*)0xe8000a)
#define CRTR_06 (*(volatile uint16_t*)0xe8000c)
#define CRTR_07 (*(volatile uint16_t*)0xe8000e)
```

src/vdp/ms_vdp.h L33-L36

What this establishes

1. MS.X directly programs the X68000 CRTC
2. The target registers are R04-R07
3. crtc60hz.c uses the same address map

[3] Official MS.X repository / src/vdp/ms_vdp.h, L33-L36.

525 lines at 31.5 kHz produces 60 Hz

$$31,500 \text{ lines/s} \div 525 \text{ lines/frame} = 60.00 \text{ frames/s}$$

HORIZONTAL RATE

31.5 kHz

Keep the line duration unchanged

FRAME TOTAL

525 lines

Sum of the four intervals

VERTICAL RATE

60.00 Hz

Frames generated each second

Vertical frequency is determined by horizontal line duration and total lines per frame.

Accumulate four interval lengths to derive R04–R07

Place 2 VSYNC, 33 back-porch, 480 visible, and 10 front-porch lines in order.

VSYNC	Back porch	Visible	Front porch
2	33	480	10

$$2 + 33 + 480 + 10 = 525 \text{ lines}$$

R05=1 → R06=34 → R07=514 → R04=524

Each register stores an interval's final position, so the value is one less than the count.

Write the MS.X-derived settings to R04–R07 in C

垂直CRTC設定

```
R04 = 0x020C; /* 525 - 1 */  
R05 = 0x0001; /* VSYNC end */  
R06 = 0x0022; /* display start */  
R07 = 0x0202; /* display end */
```

MS.X / crt60hz.c

Use the MS.X vertical timing settings to build a 525-line frame in C.

1

Start from a standard mode

IOCS mode 12 provides known 31 kHz-class horizontal timing.

2

Change vertical values only

Write the four previously derived values to R04–R07.

3

Measure after generation

Count V-DISP events and compare them with elapsed time.

MS.X supplies the setting rationale; the C program measures the generated vertical sync.

Most CRTC registers are write-only and cannot be read back

After feedback on X that reads from R04-R07 return zero, both the implementation and this guide were corrected.

Information	Readable	Treatment in sample
IOCS CRT mode number	Yes	Saved with _iocs_crtmod(-1)
CRTC R20 and R21	Yes	Not saved directly here
CRTC R04-R07	No (returns zero)	Never read back
Custom CRTC timing	Not fully available	Exact restoration not guaranteed

The corrected wording is IOCS mode re-selection, not saving R04-R07.

Sources: XEIJ CRTC.java, nono vm/crtc.cpp, GamesX CRTC Registers

The R04-R07 readback logic was removed after the feedback

BEFORE (INCORRECT)

```
for (i = 0; i < 4; ++i) {  
    saved_crtc[i] =  
        read_crtc(crtc_regs[i]);  
}
```

Real hardware returns zero for R04-R07 reads

AFTER

```
old_mode = _iocs_crtmod(-1);  
_iocs_crtmod(12);  
/* write and measure R04-R07 */  
_iocs_crtmod(old_mode);
```

Only the IOCS mode number is saved

On exit, the saved mode is passed to IOCS so its standard CRTC settings are applied again.

Save the IOCS mode, start from known mode 12, then change vertical values

CORRECTED SEQUENCE

```
old_mode = _iocs_crtmod(-1);  
_iocs_crtmod(12);  
write_crtc(R05, 0x0001);  
write_crtc(R06, 0x0022);  
write_crtc(R07, 0x0202);  
write_crtc(R04, 0x020c);  
/* after measurement */  
_iocs_crtmod(old_mode);
```

- 1 Save only the mode number
- 2 Never read back R04-R07
- 3 Re-apply the IOCS standard mode on exit

Custom R04-R07 values installed by another application before launch cannot be restored exactly.

Watch GPIP bit 4 and wait for the next V-DISP rise

Count one new display start regardless of the current signal level.

1**GPIP bit 4**

General Purpose Input/Output Port.
\$10 reflects V-DISP (0: blanking, 1: display); \$40 is CIRQ.

2**Wait for Low first**

Do not count the current High interval twice.

3**Use a timeout**

Return to restoration if the signal stops.

`WAIT_VDISP()``#define GPIP_VDISP 0x10``while ((read_gpip() & GPIP_VDISP) != 0) { }``while ((read_gpip() & GPIP_VDISP) == 0) { }``crtc60hz.c`

One `wait_vdisp()` call counts one following V-DISP start.

Read the end time immediately after V-DISP number 600

測定区間

```
wait_vdisp();
start = _iocs_ontime();

for (frame = 1; frame <= 600; ++frame) {
    wait_vdisp();
    if (frame == 600) end = _iocs_ontime();
}
```

crtc60hz.c

Align both timestamps to V-DISP boundaries.

1

Start

Read start immediately after the first boundary.

2

600 intervals

Wait for 600 additional V-DISP events.

3

End

Read end before drawing the next frame.

$\text{Hz} \times 100 = 600 \times 10000 \div \text{elapsed time in } 1/100 \text{ s}$

Every exit path re-applies the saved IOCS mode



The program returns to the standard IOCS mode; it does not reconstruct every prior CRTC value.

About 60 Hz confirmed on hardware, emulators, WebX68k, and MPX68K

Hardware

60.0 Hz

X68000 and X68000 Z

Emulators

About 60 Hz

XEij and XM6 TypeG

Web / modern

About 60 Hz

WebX68k and MPX68K

All listed environments now follow the 525-line CRTc setting; X68000 Z measured 60.0 Hz.

CRTC timing must propagate through the complete emulated-time path

A correct signal level is not enough; every consumer must share the same derived period.

1. Register writes

R00, R04, and R20 define the active timing state

2. Derived timing

Calculate line duration, total lines, and V-DISP boundaries

3. Emulated time

CPU, raster events, timers, audio, and AV metadata share it

4. Host presentation

Present completed frames without redefining guest timing

WebX68k, MPX68K, and X68000 Z now demonstrate CRTC-dependent about-60-Hz operation.

Implementation references: [2] XEIJ, [4] XM6 2.06, [5] px68k-libretro PR #1, [6] MPX68K PR #26.

XEiJ raises and lowers V-DISP at raster-derived boundaries

```
// CRTC.java L2878-L2882
if (n != crtVDispStart) {
    TickerQueue.tkqAdd (crtTicker = DuplicationOmitIdleSync,
                        crtClock += crtHFrontLength);
} else {
    MC68901.mfpVdispRise ();
}

// CRTC.java L2979-L2982
if (n != crtVIdleStart) {
    TickerQueue.tkqAdd (crtTicker = DuplicationOmitDispEvenSync,
                        crtClock += crtHFrontLength);
} else {
    MC68901.mfpVdispFall ();
}
```

Meaning

V-DISP rises at crtVDispStart and falls at crtVIdleStart.

Boundaries derived from R04-R07 drive both MFP signal changes and the next scheduled event.

[2] XEiJ / CRTC.java L2878-L2882 and L2979-L2982 (official source).

XM6 2.06 recalculates vertical timing from R04-R07

```
crtc.v_sync = ((p[4] & 0x3ff) + 1);  
crtc.v_pulse = ((p[5] & 0x3ff) + 1);  
crtc.v_back = ((p[6] & 0x3ff) + 1) - crtc.v_pulse;  
crtc.v_front = crtc.v_sync - ((p[7] & 0x3ff) + 1);
```

vm/crtc.cpp L782-L785 / CRTC::ReCalc()

What the public source shows

R04 supplies the total raster count. R05-R07 define boundaries used to recompute vertical sync and both porches.

This quotation is from the public XM6 2.06 source, not XM6 TypeG.

[4] XM6 2.06 public source / vm/crtc.cpp L782-L785.

Before the fix, CRTC-derived timing did not reach every WebX68k subsystem

Timing path before the fix

CRTC R00 / R04 / R20
|
derived timing
X
CPU / timer / audio / AV

1

Timing was disconnected

Some paths still used a fixed period after CRTC writes.

2

Subsystems disagreed

CPU, raster, timer, audio, and AV paths did not share one period.

3

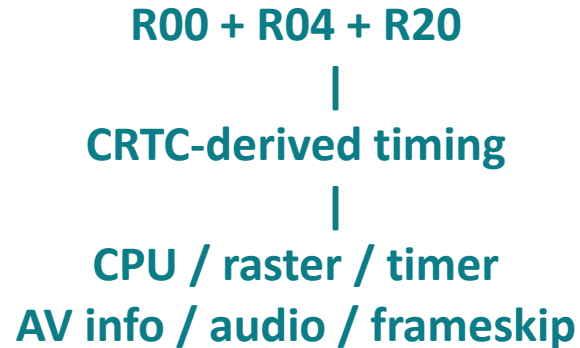
Observed result

The 525-line setting did not produce about 60 Hz in WebX68k.

The defect crossed multiple time paths; it was not limited to image presentation.

px68k recalculates CRTC timing and shares it across every time path

Merged timing path



1

Calculate from CRTC state

Derive the current frame period from horizontal and vertical timing.

2

Share one period

CPU, raster, timers, AV, audio, and frameskip use the same value.

3

Track writes and loads

Recalculate after register writes and after loading older save states.

After PR #1 was merged, crtc60hz.x ran at about 60 Hz in WebX68k.

px68k uses one CRTC-derived timing source

x68k/crtc.c L255-L266

```
uint32_t CRTC_GetFrameClocks(void)
{
    uint32_t oscillator, divider, columns;
    uint64_t clocks;

    if (VLINE_TOTAL != 0 &&
        CRTC_GetHorizontalTiming(&oscillator, &divider, &columns))
    {
        const uint64_t numerator = CRTC_TIMING_CLOCK * divider * 8U *
            columns * VLINE_TOTAL;
        clocks = (numerator + oscillator / 2U) / oscillator;
    }
}
```

1

Calculate in the core

Use R00, R04, and R20 for horizontal and vertical timing.

2

Share the result

Feed CPU, raster, timer, AV, audio, and frameskip paths.

3

Defend against invalid data

Uses 64-bit math; range limits are applied at L274-L282 in the same file.

The current CRTC state, not a fixed fps constant, is the single timing authority.

CONCLUSION

Generated and measured about 60 Hz in 31 kHz mode, then confirmed it across current environments

- 1 Derived the 525-line values from the four vertical intervals and R04-R07.
- 2 Never read back R04-R07; re-select the saved IOCS standard mode on exit.
- 3 Confirmed about 60 Hz in WebX68k, MPX68K, and X68000 Z.

Acknowledgments

This investigation benefited from the public documentation and implementations of XEiJ, MS.X, and XM6.

We thank the contributor who identified the R04-R07 readback limitation, and the WebX68k developer for the review, save-state compatibility feedback, and merge of the correction.

We also thank the MPX68K developer for implementing CRTC-derived timing and confirming 60.00 Hz with crtc60hz.

The feedback led us to revise the implementation and documentation for greater accuracy.

References

Numbered primary sources and directly relevant implementation records

[1] Kunichiko, X68000用MSXシミュレータ「MS.X」開発秘話, Japanese e-book v1.0 (2026-08-16)
<https://booth.pm/ja/items/8672923>

[2] XEIJ / CRTC.java - CRTC registers and raster-event timing
<https://stdkmd.net/xeij/source/xeij-CRTC.java.htm>

[3] Official MS.X source / src/vdp/ms_vdp.h
https://github.com/kunichiko/X68000_MSX_Simulator/blob/main/src/vdp/ms_vdp.h#L33-L36

[4] XM6 2.06 public source / vm/crtc.cpp
http://retropc.net/pi/xm6/xm6_206s.zip

[5] px68k-libretro PR #1 - CRTC timing fix
<https://github.com/uraraworks/px68k-libretro/pull/1>

[6] MPX68K PR #26 - CRTC timing and 60.00 Hz verification
<https://github.com/YosAwed/MPX68K/pull/26>

[7] crtc60hz - public source for the sample explained in this guide
<https://github.com/renatus-novus-x/crtc60hz>

URLs checked on 2026-08-19. English quotations from [1] are identified as translations.